Journal of Combinatorial Optimization (2023) 46:9
https://doi.org/10.1007/s10878-023-01074-x

Scheduling problems with rejection to minimize the k-th power of the makespan plus the total rejection cost

Lingfa Lu[1] · **Liqi Zhang**[2]

Accepted: 25 July 2023 / Published online: 17 August 2023

Abstract
In this paper, we consider several scheduling problems with rejection on $m \geq 1$ identical machines. Each job is either accepted and processed on the machines, or it is rejected by paying a certain rejection cost. The objective is to minimize the sum of the k-th power of the makespan of accepted jobs and the total rejection cost of rejected jobs, where $k > 0$ is a given constant. We also introduce the conception of "job splitting" in our problems. First, we consider the single machine scheduling problem, i.e., $m = 1$. When job splitting is allowed, we propose an $O(n \log n)$-time optimal algorithm for the problem. When job splitting is not allowed, we show that this problem is polynomially solvable when $k \in (0, 1]$ and it becomes binary NP-hard when $k > 1$. Furthermore, for the NP-hard problem, we propose a pseudo-polynomial dynamic programming algorithm and a fully polynomial-time approximation scheme (FPTAS). Finally, we also extend our problems and some results to $m \geq 2$ identical parallel machines.

Keywords Scheduling with rejection · NP-hard · Dynamic programming · FPTAS

1 Introduction

In recent 70 years, the rich variety of scheduling problems have been widely studied in many manufacturing and service industries. In most situations, the manufacturers or the service providers hope to finish all jobs (orders or services) in the shortest time.

A preliminary version of this paper appears in Computing and Combinatorics Conference (COCOON 2022). Lecture Notes in Computer Science, vol. 13595, Springer, Shenzhen, China.

✉ Lingfa Lu
lulingfa@zzu.edu.cn

[1] School of Mathematics and Statistics, Zhengzhou University, Zhengzhou, Henan, People's Republic of China

[2] College of Information and Management Science, Henan Agricultural University, Zhengzhou, Henan, People's Republic of China

Thus, the makespan $C_{\max}$ is the most basic and well-studied objective function in the literature. In this paper, we consider a new objective function $\left(C_{\max}\right)^k$, i.e., the k-th power of the makespan. Clearly, $\left(C_{\max}\right)^k$ is an extension of the makespan $C_{\max}$, which reflects the more general cost on the energy assumption especially when the energy price varies over time.

1.1 Scheduling to minimize the *k*-th power of the makespan

To the best of our knowledge, $\left(C_{\max}\right)^k$ is rarely studied in the literature. The earliest paper studied $\left(C_{\max}\right)^k$ as a part of the objective function is probably the paper by Ishii and Nishida (1986) in 1986. This paper considered the two-machine open-shop scheduling problem with controllable machine speeds, where the machine speeds s_1, s_2 are not fixed but variables. The task is to determine the optimal machine speeds and an optimal schedule such that $\left(C_{\max}\right)^k + c_1 s_1^q + c_2 s_2^q$ is minimized, where c_1, c_2 are positive constants, and k, q are positive integers. For this problem, Ishii and Nishida (1986) presented an $O(n \log n)$-time algorithm which finds the optimal speeds of two machines and the corresponding optimal schedule. Ishii et al. (1987) further studied a similar problem in the two-machine mixed-shop environment. They provided an $O(n^2 \log n)$-time algorithm to determine the optimal speeds and the corresponding optimal schedule. In addition, Shakhlevich and Strusevich (2006) considered the single machine scheduling problem with the controllable processing times and/or the controllable release dates. The task is to determine the optimal speeds v_r, v_p and an optimal schedule such that $\left(C_{\max}\right)^k + c_r v_r^q + c_p v_p^q$ is minimized, where c_r, c_p are positive constants. They also developed a polynomial-time algorithm for this problem.

1.2 Scheduling with rejection

Scheduling problems with rejection have been studied extensively in the recent two decades. An important application of scheduling with rejection arises in high loaded make-to-order production systems with limited capacity and tight requirements. Another important application occurs in scheduling with an outsourcing option. Moreover, there are many close connections between scheduling with rejection and other fields such as scheduling with controllable processing times, scheduling with costs, and scheduling with due date assignment, etc.

The notion of "scheduling with rejection" was first introduced by Bartal et al. (2000) in 2000. This paper studied the off-line and on-line problems on m identical parallel machines. The objective is to minimize the sum of the makespan of the accepted jobs and the total rejection penalty of the rejected jobs. For the on-line problem, they proposed a best-possible on-line algorithm with the competitive ratio $\frac{\sqrt{5}+3}{2} \approx 2.618$; for the off-line problem, they provided a polynomial-time approximation scheme (PTAS) for arbitrary m and a fully polynomial-time approximation scheme (FPTAS) for fixed m, respectively. Hoogeveen et al. (2003) studied the preemptive scheduling problems on m (identical, uniformly related, or unrelated) parallel machines. They provided

a complete classification of these scheduling problems with respect to complexity and approximability. Zhang et al. (2009) considered the single machine scheduling problem with release dates and rejection. They showed that this problem is NP-hard and presented an FPTAS for this problem. For more research results on this topic, the readers can refer to two surveys provided by Shabtay et al. (2013) and Zhang (2020), respectively. More recent papers on scheduling with rejection are Atsmony and Mosheiov (2023), Chen and Li (2020), Hermelin et al. (2019), Koulamas and Kyparisis (2023), Liu and Lu (2020), Liu (2020), Ma and Guo (2021), Mor et al. (2020), Mor and Shabtay (2022) and Oron (2021).

2 Problem formulation

Scheduling problems with rejection can be stated as follows: there are n jobs $J_1, J_2, \ldots, J_n$ and m identical machines $M_1, M_2, \ldots, M_m$. Each job has a processing time p_j and a rejection cost e_j. Here we assume that all parameters are some positive integers. Each job J_j is either accepted and then processed on the machines, or it is rejected by paying the corresponding rejection cost e_j. Without any loss of generality, we assume that all jobs have been resorted such that $\frac{e_1}{p_1} \geq \frac{e_2}{p_2} \geq \cdots \geq \frac{e_n}{p_n}$. Let A and R be the set of the accepted jobs and the set of the rejected jobs, respectively. Furthermore, let π be a feasible schedule for the jobs in A and let C_j be the completion time of J_j in π, where $J_j \in A$. We define $C_{\max} = \max\{C_j : J_j \in A\}$ and $\left(C_{\max}\right)^k$ as the makespan and the k-th power of the makespan, respectively. The objective is to minimize the sum of the k-th power of the makespan of accepted jobs and the total rejection cost of rejected jobs. Using the general notation for scheduling problems, our problem can be denoted by $Pm||\left(C_{\max}\right)^k + \sum_{J_j \in R} e_j$. When $m = 1$, the corresponding single machine scheduling problem is denoted by $1||\left(C_{\max}\right)^k + \sum_{J_j \in R} e_j$.

Note that, when $k \leq 0$, problem $Pm||\left(C_{\max}\right)^k + \sum_{J_j \in R} e_j$ is trivial since all jobs are accepted and processed on the same machine in the optimal schedule. Furthermore, when $k = 1$, problem $Pm||C_{\max} + \sum_{J_j \in R} e_j$ has been studied by Bartal et al. (2000) in the literature. Thus, we assume that $k \in (0, 1) \cup (1, \infty)$ in this paper.

In order to design some exact or approximation algorithms, we introduce the conception of "job splitting" into our scheduling problems. In detail, each job J_j can be split arbitrarily to $q > 1$ parts $J_j^1, J_j^2, \ldots, J_j^q$ such that $\sum_{i=1}^q p_j^i = p_j$, $\sum_{i=1}^q e_j^i = e_j$ and $e_j^i = \frac{e_j}{p_j} \cdot p_j^i$, where p_j^i and e_j^i are the corresponding processing time and rejection cost of the split part J_j^i, $i = 1, \ldots, q$. All split parts $J_j^1, J_j^2, \ldots, J_j^q$ of J_j can be viewed as q independent jobs so that they can be accepted or rejected independently. Furthermore, all accepted parts can be processed in parallel, i.e., some overlaps are allowed in the processing of these accepted parts. Obviously, "job splitting" is a more stronger concept than "job preemption" in the classical scheduling theory. In fact, there are many applications of job splitting in practice. Some scheduling problems with job splitting has been considered in the literature (see Xing and Zhang (2000)). When job splitting is allowed, the corresponding scheduling problems can be denoted by $Pm|\text{split}|\left(C_{\max}\right)^k + \sum_{J_j \in R} e_j$ and $1|\text{split}|\left(C_{\max}\right)^k + \sum_{J_j \in R} e_j$, respectively.

When $m = 1$, we can assume that each job J_j can be split into two parts J_j^A and J_j^R, then we have $p_j^A + p_j^R = p_j$ and $\frac{e_j^A}{p_j^A} = \frac{e_j^R}{p_j^R} = \frac{e_j}{p_j}$. Furthermore, J_j^A is accepted and processed on the machine, and J_j^R is rejected by paying the rejection cost e_j^R. If we view p_j^A, p_j^R and e_j^R as the actual processing time t_j, the compression amount u_j and the compression cost $x_j u_j$ of job J_j, i.e., $p_j^A = t_j = p_j - u_j$, $p_j^R = u_j$ and $e_j^R = x_j u_j = \frac{e_j}{p_j} u_j$, then scheduling problems with job splitting and rejection are in fact equivalent to the corresponding scheduling problems with controllable processing times. Thus, problem $1|\text{split}|\left(C_{\max}\right)^k + \sum_{J_j \in R} e_j$ is equivalent to $1|t_j = p_j - u_j|\left(C_{\max}\right)^k + \sum_{j=1}^n x_j u_j$. For the Pareto-optimization problem $1|t_j = p_j - u_j|\#(f_{\max}, \sum_{j=1}^n x_j u_j)$, Van Wassenhove and Baker (1982) presented an efficient $O(n^2)$-time algorithm for finding all Pareto-optimization points when the functions $f_j(t)$ satisfy the condition that there exists a permutation π such that $f_{\pi(1)}(t) \leq f_{\pi(2)}(t) \leq \cdots \leq f_{\pi(n)}(t)$ for all t, where $f_j(t)$ is the cost function of J_j when the completion time of J_j is t. Clearly, in our problems, $f_j(t) = t^k$ holds for each $j = 1, \ldots, n$ and it satisfies the above condition. Thus, by applying the method in Van Wassenhove and Baker (1982) and selecting the minimum value of $\left(C_{\max}\right)^k + \sum_{J_j \in R} e_j$ among all Pareto-optimization points, problem $1|\text{split}|\left(C_{\max}\right)^k + \sum_{J_j \in R} e_j$ can be solved in $O(n^2)$ time. Note further that the worst-case running time of the method in Van Wassenhove and Baker (1982) for problem $1|\text{split}|\left(C_{\max}\right)^k + \sum_{J_j \in R} e_j$ is exactly $O(n^2)$. For the more results on scheduling with controllable processing times, we refer the readers to two surveys provided by Nowicki and Zdrzalka (1990) and Shabtay and Steiner (2007).

The rest of the paper is organized as follows. In Sect. 3, we consider the problem $1|\text{split}|\left(C_{\max}\right)^k + \sum_{J_j \in R} e_j$ and provide an $O(n \log n)$-time optimal schedule for this problem. In Sect. 4, we consider the problem $1||\left(C_{\max}\right)^k + \sum_{J_j \in R} e_j$. We show that this problem is polynomially solvable when $k \in (0, 1)$ and it becomes binary NP-hard when $k > 1$. For the NP-hard problem, we propose a pseudo-polynomial dynamic programming algorithm and a fully polynomial-time approximation scheme (FPTAS) for this problem. In Sect. 5, we consider the problem $Pm|\text{split}|\left(C_{\max}\right)^k + \sum_{J_j \in R} e_j$ and transform it into a corresponding single machine problem. That is, problem $Pm|\text{split}|\left(C_{\max}\right)^k + \sum_{J_j \in R} e_j$ can be solved in $O(n \log n)$ time by borrowing the proposed algorithm in Sect. 3. In Sect. 6, we consider the problem $Pm||\left(C_{\max}\right)^k + \sum_{J_j \in R} e_j$ and present a dynamic programming algorithm for it. Finally, some conclusions and future research are provided in Sect. 7.

3 Problem $1|\text{split}|\left(C_{\max}\right)^k + \sum_{J_j \in R} e_j$

In this section, we consider the problem $1|\text{split}|\left(C_{\max}\right)^k + \sum_{J_j \in R} e_j$. As stated above, Van Wassenhove and Baker (1982) presented an $O(n^2)$-time algorithm to solve a more general problem. We will propose an improved algorithm to solve this problem in $O(n \log n)$ time. Assume that all jobs $J_1, J_2, \ldots, J_n$ have been resorted such that $\frac{e_1}{p_1} \geq \frac{e_2}{p_2} \geq \cdots \geq \frac{e_n}{p_n}$. We have the following lemma.

Lemma 1 *For problem $1|split|\left(C_{\max}\right)^k + \sum_{J_j \in R} e_j$, there exists a job J_{j^*} such that $J_1, \ldots, J_{j^*-1}$ are fully accepted and $J_{j^*+1}, \ldots, J_n$ are fully rejected in an optimal schedule π^*. That is, J_{j^*} is the unique job which is possible be split in π^*.*

Proof Assume that π^* is an optimal schedule for problem $1|\text{split}|\left(C_{\max}\right)^k + \sum_{J_j \in R} e_j$. Let A^* and R^* denote the set of accepted jobs (or split parts) and the set of rejected jobs (or split parts) in π^*. If $A^* = \emptyset$, then $J_{j^*} = J_1$ is the required job. Thus, we assume that $A^* \neq \emptyset$ in the following. Furthermore, let $C^*_{\max}$ be the optimal makespan in π^*. Thus, we have

$$Z^* = (C^*_{\max})^k + \sum_{J_j \in R^*} e_j = (C^*_{\max})^k + \sum_{j=1}^{n} e_j - \sum_{J_j \in A^*} e_j.$$

Thus, if $C^*_{\max}$ is known, the remaining problem is to maximize $\sum_{J_j \in A^*} e_j$ under the constraint $\sum_{J_j \in A^*} p_j = C^*_{\max}$. Note that the latter problem is equivalent to the classical **Fractional Knapsack problem**, where the Fractional Knapsack problem is defined as follows: There are a knapsack with capacity W and n items with weights $w_1, \ldots, w_n$ and profits $c_1, \ldots, c_n$, respectively. The task is to find numbers $x_1, \ldots, x_n \in [0, 1]$ such that $\sum_{j=1}^{n} x_j w_j \leq W$ and $\sum_{j=1}^{n} x_j c_j$ is maximum (see Korte and Vygen (2018)). Notice that there is a simple $O(n \log n)$-time algorithm for the Fractional Knapsack problem. Thus, if $C^*_{\max}$ is known, problem $1|\text{split}|\left(C_{\max}\right)^k + \sum_{J_j \in R} e_j$ can be solved by the following procedure.

(1) Resort all jobs such that $\frac{e_1}{p_1} \geq \frac{e_2}{p_2} \geq \cdots \geq \frac{e_n}{p_n}$. Set

$$j^* := \min \left\{ j \in \{1, \ldots, n\} : \sum_{1 \leq i \leq j} p_i \geq C^*_{\max} \right\}.$$

(2) Set

$$\begin{cases} y_j := 1 & \text{for } j = 1, \ldots, j^* - 1, \\ y_{j^*} := \frac{C^*_{\max} - \sum_{1 \leq j \leq j^*-1} p_j}{p_{j^*}}, & \\ y_j := 0 & \text{for } j = j^* + 1, \ldots, n. \end{cases}$$

(3) Set $p^A_j = y_j p_j$ and $e^R_j = (1 - y_j) e_j$. All split parts J^A_j are processed in an arbitrary order on the machine and reject all split parts J^R_j.

Note that in the above algorithm, $J_1, \ldots, J_{j^*-1}$ are accepted fully and $J_{j^*+1}, \ldots, J_n$ are rejected fully in π^*. Furthermore, J_{j^*} is the unique job which is possible be split in π^*. This completes the proof of Lemma 1. □

From the above discussion, we can see that $1|\text{split}|\left(C_{\max}\right)^k + \sum_{J_j \in R} e_j$ can be solved once $C^*_{\max}$ is determined. However, we do not know $C^*_{\max}$ in advance. Now, we assume that J_{j^*} has been determined, but $C^*_{\max}$ is not known in advance. It follows that $J_1, \ldots, J_{j^*-1}$ are accepted fully and $J_{j^*+1}, \ldots, J_n$ are rejected fully in π^*. Let Z^*_s be the optimal objective value for problem $1|\text{split}|\left(C_{\max}\right)^k + \sum_{J_j \in R} e_j$. Thus, we have

$$Z^*_s = (C^*_{\max})^k + \sum_{J_j \in R^*} e_j = \left(\sum_{j=1}^{j^*-1} p_j + y_{j^*} p_{j^*}\right)^k + (1 - y_{j^*}) e_{j^*} + \sum_{j=j^*+1}^{n} e_j. \tag{1}$$

Since $\sum_{j=1}^{j^*-1} p_j$ and $\sum_{j=j^*+1}^{n} e_j$ are fixed, Z^*_s is a function of the variable y_{j^*}. Set $f(y_{j^*}) = \left(\sum_{j=1}^{j^*-1} p_j + y_{j^*} p_{j^*}\right)^k + (1 - y_{j^*}) e_{j^*} + \sum_{j=j^*+1}^{n} e_j$. Clearly, we have

$$f'(y_{j^*}) = k \left(\sum_{j=1}^{j^*-1} p_j + y_{j^*} p_{j^*}\right)^{k-1} p_{j^*} - e_{j^*} \tag{2}$$

and

$$f''(y_{j^*}) = k(k-1) \left(\sum_{j=1}^{j^*-1} p_j + y_{j^*} p_{j^*}\right)^{k-2} \left(p_{j^*}\right)^2, \tag{3}$$

where $f'(y_{j^*})$ and $f''(y_{j^*})$ are the first derivative and the second derivative of $f(y^*_j)$, respectively.

Clearly, when $0 < k < 1$, we have $f''(y_{j^*}) \leq 0$. It follows that $Z^*_s = f(y_{j^*})$ is a concave ($\bigcap$-shaped) function on y_{j^*}. Thus, the minimum value of $Z^*_s = f(y_{j^*})$ must be reached at two endpoints, i.e., $y_{j^*} = 0$ or $y_{j^*} = 1$. Note that, in this situation, no job can be split in π^*. Thus, we have the following algorithm to solve the problem $1|\text{split}|\left(C_{\max}\right)^k + \sum_{J_j \in R} e_j$ with $k \in (0, 1)$. The detailed algorithm is as follows:

Algorithm A_1

Step 1 Sort all jobs $J_1, J_2, \ldots, J_n$ such that $\frac{e_1}{p_1} \geq \frac{e_2}{p_2} \geq \cdots \geq \frac{e_n}{p_n}$.

Step 2 Set $S(0) = \emptyset$ and $S(j) = \{J_1, \ldots, J_j\}$ for each $j = 1, \ldots, n$. Furthermore, we also set $\overline{S(0)} = \{J_1, \ldots, J_n\}$ and $\overline{S(j)} = \{J_{j+1}, \ldots, J_n\}$. Furthermore, we generate $n + 1$ schedules by accepting the jobs in $S(j)$ and rejecting the jobs in $\overline{S(j)}$ for each $j = 0, 1, \ldots, n$. The obtained schedules are denoted by $\pi(0), \pi(1), \ldots, \pi(n)$, respectively.

Step 3 For each $j = 0, 1, \ldots, n$, we compute the objective value $Z(j)$ of schedule $\pi(j)$ by the equation $Z(j) = (\sum_{i=1}^{j} p_i)^k + \sum_{i=j+1}^{n} e_i$.

Table 1 A numerical example for problem $1|\text{split}|\sqrt{C_{\max}} + \sum_{J_j \in R} e_j$

Jobs (J_j)	J_1	J_2	J_3	J_4	J_5	J_6	J_7
Processing times (p_j)	4	21	39	80	81	175	225
Rejection costs (e_j)	10	8	7	5	2	2	1

Step 4 Select the schedule π with $\pi \in \{\pi(j) : j = 0, 1, \ldots, n\}$ such that $Z(\pi) = \min\{Z(j) : j = 0, 1, \ldots, n\}$.

Theorem 1 *Algorithm A_1 finds an optimal schedule in $O(n \log n)$ time for problem $1|split|\left(C_{\max}\right)^k + \sum_{J_j \in R} e_j$ with $k \in (0, 1)$.*

Proof The correctness of algorithm A_1 is guaranteed by the above discussion. Step 1 costs an $O(n \log n)$ time. Furthermore, we can generate the $n + 1$ schedules in $O(n)$ time and compute the corresponding objective values in $O(n)$ time. Finally, it needs an $O(n)$ time to find the schedule with the smallest objective value. Thus, the time complexity of Algorithm A_1 is $O(n \log n)$. □

Next, we present a numerical example to illustrate the working of Algorithm A_1 for problem $1|\text{split}|\sqrt{C_{\max}} + \sum_{J_j \in R} e_j$, i.e., $k = \frac{1}{2}$. Consider the job instance with the details in Table 1.

Note that all jobs $J_1, \ldots, J_7$ have satisfied the condition $\frac{e_1}{p_1} \geq \frac{e_2}{p_2} \geq \cdots \geq \frac{e_7}{p_7}$. Furthermore, there is no job which is split in an optimal schedule. It is not hard to verify that, the optimal schedule can be obtained by accepting the jobs J_1, J_2, J_3, J_4 and rejecting the jobs J_5, J_6, J_7. Thus, the optimal objective value is $Z_s^* = \sqrt{C_{\max}} + \sum_{J_j \in R} e_j = \sqrt{144} + 5 = 17$.

When $k > 1$, we have $f''(y_{j^*}) > 0$ and then $Z_s^* = f(y_{j^*})$ is a convex ($\bigcup$-shaped) function on y_{j^*}. We denote y by the unique root of equation $f'(y) = 0$. Here we assume that the unique root y of $f'(y_{j^*}) = 0$ and the corresponding function value $f(y)$ can be computed in a constant time. Clearly, if $y \notin (0, 1)$, then $Z_s^* = f(y_{j^*})$ can reach its minimum value at two endpoints, i.e., $y_{j^*} = 0$ or $y_{j^*} = 1$. If $y \in (0, 1)$, then $Z_s^* = f(y_{j^*})$ can reach its minimum value at $y_{j^*} = y$.

From the above discussion, if J_{j^*} is determined, we can find an optimal schedule for problem $1|\text{split}|\left(C_{\max}\right)^k + \sum_{J_j \in R} e_j$ with $k > 1$. Note that there are at most n choices for job J_{j^*}. Thus, by enumerating all n choices for job J_{j^*}, we can find an optimal schedule for problem $1|\text{split}|\left(C_{\max}\right)^k + \sum_{J_j \in R} e_j$ with $k > 1$. The detailed algorithm for problem $1|\text{split}|\left(C_{\max}\right)^k + \sum_{J_j \in R} e_j$ with $k > 1$ is as follows:

Algorithm A_2

Step 1 Sort all jobs $J_1, J_2, \ldots, J_n$ such that $\frac{e_1}{p_1} \geq \frac{e_2}{p_2} \geq \cdots \geq \frac{e_n}{p_n}$. For each $j^* = 1, \ldots, n$, set $f(y_{j^*}) = (\sum_{j=1}^{j^*-1} p_j + y_{j^*} p_{j^*})^k + (1 - y_{j^*}) e_{j^*} + \sum_{j=j^*+1}^{n} e_j$.

Step 2 Compute the minimum value of $f(y_{j^*})$ by the previous analysis, where $y_{j^*} \in [0, 1]$. Without loss of generality, we assume $Z^* = f(y_{j^*})$ can reach its minimum value at $y_{j^*} = y_j$, where $y_j = 0$, or $y_j = 1$, or $y \in (0, 1)$ is the unique root of the equation $f'(y_{j^*}) = 0$.

Table 2 A numerical example for problem $1|\text{split}|\left(C_{\max}\right)^2+\sum_{J_j\in R} e_j$

Jobs (J_j)	J_1	J_2	J_3	J_4	J_5	J_6	J_7
Processing times (p_j)	2	2	4	6	8	8	10
Rejection costs (e_j)	200	180	150	120	100	70	50

Step 3 Accept the jobs $J_1,\ldots,J_{j^*-1}$ and a split part $J_{j^*}^A$ of J_{j^*} with the length of $p_{j^*}^A = y_j p_j$. Furthermore, we reject the remaining part of J_{j^*} and the jobs $J_{j^*+1},\ldots,J_n$. Let $\pi(j^*)$ be the obtained schedule and $Z(j^*)$ be the corresponding objective function value.

Step 4 Select the schedule $\pi\in\{\pi(j^*): j^*=1,\ldots,n\}$ with $Z(\pi)=\min\{Z(j^*): j^*=1,\ldots,n\}$.

Theorem 2 *Algorithm A_2 finds an optimal schedule in $O(n\log n)$ time for problem $1|split|\left(C_{\max}\right)^k+\sum_{J_j\in R} e_j$ with $k>1$.*

Proof The correctness of algorithm A_2 is guaranteed by the above discussion. Step 1 costs an $O(n\log n)$ time. Furthermore, for each $j^*=1,\ldots,n$, the root of $f'(y_{j^*})=0$ and the minimum value $f(y_{j^*})$ can be computed in a constant time. Thus, the overall time complexity of Algorithm A_2 is still $O(n\log n)$. □

Furthermore, we also use a numerical example to illustrate the working of Algorithm A_2 for problem $1|\text{split}|\left(C_{\max}\right)^2+\sum_{J_j\in R} e_j$, i.e., $k=2$. Consider the job instance with the details in Table 2.

Note that all jobs $J_1,\ldots,J_7$ have satisfied the condition $\frac{e_1}{p_1}\geq\frac{e_2}{p_2}\geq\cdots\geq\frac{e_7}{p_7}$. It is not hard to verify that J_4 is the split job in the optimal schedule. Furthermore, we have

$$f(y_4)=(8+6y_4)^2+120(1-y_4)+220$$

and $f'(y_4)=12(8+6y_4)-120$. It follows that $y_4=\frac{1}{3}$ is the unique root of $f'(y_4)=0$. That is, in the optimal schedule, jobs J_1, J_2, J_3 and a split part J_4^A with $p_4^A=6y_4=2$ are accepted, s split part J_4^R with $p_4^R=6(1-y_4)=4$ and jobs J_5, J_6, J_7 are rejected. Thus, the optimal objective value is $Z_s^*=\left(C_{\max}\right)^2+\sum_{J_j\in R} e_j=10^2+80+220=400$.

4 Problem $1||\left(C_{\max}\right)^k+\sum_{J_j\in R} e_j$

Note that, when $k\in(0,1)$, problem $1|\text{split}|\left(C_{\max}\right)^k+\sum_{J_j\in R} e_j$ can be solved by Algorithm A_1. Note further that, no job can be split in the optimal schedule for problem $1|\text{split}|\left(C_{\max}\right)^k+\sum_{J_j\in R} e_j$. This also implies that problem $1||\left(C_{\max}\right)^k+\sum_{J_j\in R} e_j$ with $k\in(0,1)$ can be solved by Algorithm A_1 in $O(n\log n)$ time.

Next, we only consider the problem $1||\left(C_{\max}\right)^k+\sum_{J_j\in R} e_j$ with $k>1$. First, we show that this problem is NP-hard when $k>1$. This implies that, though problems

$1||\left(C_{\max}\right)^k + \sum_{J_j \in R} e_j$ with $k \leq 1$ and $1||\left(C_{\max}\right)^k + \sum_{J_j \in R} e_j$ with $k > 1$ are very similar, they are different in the computational complexity.

4.1 NP-hardness proof when $k > 1$

We use the classical NP-complete Partition problem (see Garey and Johnson (1979)) for the reduction.

Partition problem: Given $t+1$ integers $a_1, a_2, \ldots, a_t$, B such that $\sum_{i=1}^{t} a_i = 2B$, is there a set S with $S \subset \{1, 2, \ldots, t\}$ such that $\sum_{i \in S} a_i = B$?

Theorem 3 *Problem* $1||\left(C_{\max}\right)^k + \sum_{J_j \in R} e_j$ *with* $k > 1$ *is NP-hard.*

Proof For a given instance of the Partition problem, an instance of the decision version of $1||\left(C_{\max}\right)^k + \sum_{J_j \in R} e_j$ is constructed as follows.

- There are $n = t$ jobs.
- For $1 \leq j \leq t$, we have a job J_j with $p_j = a_j$ and $e_j = kB^{k-1}a_j$;
- The threshold value is defined by $Y = (k+1)B^k$.
- The decision version asks whether there is a schedule π such that $\left(C_{\max}\right)^k + \sum_{J_j \in R} e_j \leq Y$.

It can be seen that the above construction can be done in polynomial time. Set $x = C_{\max} = \sum_{J_j \in A} p_j = \sum_{J_j \in A} a_j$. Thus, we have $\left(C_{\max}\right)^k = x^k = \left(\sum_{J_j \in A} a_j\right)^k$. It follows that

$$\begin{aligned}\left(C_{\max}\right)^k + \sum_{J_j \in R} e_j &= \left(\sum_{J_j \in A} p_j\right)^k + \sum_{J_j \in R} e_j \\ &= \left(\sum_{J_j \in A} a_j\right)^k + kB^{k-1} \sum_{J_j \in R} a_j \\ &= \left(\sum_{J_j \in A} a_j\right)^k + kB^{k-1}\left(2B - \sum_{J_j \in A} a_j\right) \\ &= x^k - kB^{k-1}x + 2kB^k.\end{aligned}$$

Set $f(x) = x^k - kB^{k-1}x + 2kB^k$ with $0 \leq x \leq 2B$. Thus, we have $x = B$ if and only if $f'(x) = kx^{k-1} - kB^{k-1} = 0$. Furthermore, we also have $f''(B) = k(k-1)B^{k-2} > 0$. Clearly, $f(x)$ can reach its minimum value $(k+1)B^k = Y$ if and only if $x = B$. Thus, there is a schedule π such that $\left(C_{\max}\right)^k + \sum_{J_j \in R} e_j \leq Y$ if and only if the Partition problem has a solution. Theorem 3 follows. □

4.2 Dynamic programming algorithm

Let $f_j(E)$ be the minimum makespan value when the jobs in consideration are $J_1, \ldots, J_j$ and the rejection cost of the rejected jobs is exactly E. Now, we consider any optimal schedule for the jobs $J_1, \ldots, J_j$ in which the rejection cost of the rejected jobs is exactly E. In such a schedule, there are two possible cases: either job J_j is rejected or job J_j is accepted.

Case 1. Job J_j is rejected. Then total rejection cost of the rejected jobs among $J_1, \ldots, J_{j-1}$ is $E - e_j$. Thus, we have $f_j(E) = f_{j-1}(E - e_j)$.

Case 2. Job J_j is accepted. Before J_j is accepted, the total rejection cost of the rejected jobs among $J_1, \ldots, J_{j-1}$ is still E. Thus, we have $f_j(E) = f_{j-1}(E) + p_j$.

Combining the above two cases, we have the dynamic programming algorithm DP1.

Dynamic programming algorithm DP1

The boundary conditions:

$$f_1(E) = \begin{cases} p_1, & \text{if } E = 0; \\ 0, & \text{if } E = e_1; \\ +\infty, & \text{otherwise.} \end{cases}$$

The recursive function:

$$f_j(E) = \begin{cases} f_{j-1}(E) + p_j, & \text{if } E < e_j; \\ \min\{f_{j-1}(E - e_j),\ f_{j-1}(E) + p_j\}, & \text{if } E \geq e_j. \end{cases}$$

The optimal value is given by $Z^* = \min\left\{(f_n(E))^k + E : 0 \leq E \leq \sum_{j=1}^{n} e_j\right\}$.

Theorem 4 *Algorithm DP1 solves problem* $1||\left(C_{\max}\right)^k + \sum_{J_j \in R} e_j$ *with* $k > 1$ *in* $O(n \sum_{j=1}^{n} e_j)$ *time.*

Next, we still use the numerical example in Table 2 to illustrate the working of algorithm DP1 for problem $1||\left(C_{\max}\right)^2 + \sum_{J_j \in R} e_j$. It is not hard to verify that $Z^* = (f_7(340))^2 + 340 = 8^2 + 340 = 404$. That is, jobs $J_1, \ldots, J_3$ are accepted and other jobs are rejected in the optimal schedule.

4.3 Approximation algorithms

In this subsection, based on the optimal schedule for problem $1|\text{split}|\left(C_{\max}\right)^k + \sum_{J_j \in R} e_j$, we propose an approximation algorithm A_3 for problem $1||\left(C_{\max}\right)^k + \sum_{J_j \in R} e_j$. Furthermore, based on the dynamic programming algorithm and the rounding technique, we obtain a fully polynomial-time approximation scheme (FPTAS) for problem $1||\left(C_{\max}\right)^k + \sum_{J_j \in R} e_j$. The approximation algorithm A_3 is as follows:

Algorithm A_3

Step 1: Assume that σ^* is an optimal schedule for problem $1|\text{split}|\left(C_{\max}\right)^k + \sum_{J_j \in R} e_j$, which can be obtained by algorithm A_2. If no job is split in σ^*, then we output the schedule σ^*. Otherwise, we assume that J_{j^*} is the unique job which is split in σ^*. Furthermore, $J_1, \ldots, J_{j^*-1}$ are accepted fully and $J_{j^*+1}, \ldots, J_n$ are rejected fully in σ^*.

Step 2: Let $y_{j^*} = \frac{p_{j^*}^A}{p_{j^*}}$ be the percentage of accepted part of J_{j^*} in σ^*. We also assume that ρ with $0 < \rho < 1$ is the unique root of $x^k + x - 1 = 0$. If $y_{j^*} \geq \rho$, then we accept $J_1, \ldots, J_{j^*}$ and reject other jobs. Otherwise, we accept $J_1, \ldots, J_{j^*-1}$ and

reject other jobs. All accepted jobs are processed consecutively in arbitrary order on the machine.

Note that, in Step 1, σ^* can be obtained by algorithm A_2 in $O(n \log n)$ time. Note further that, it takes an $O(n)$ time to obtain a feasible schedule in Step 2. Thus, the running time of Algorithm A_3 is still $O(n \log n)$. Set $f(x) = x^k + x - 1$ and $0 \leq x \leq 1$. Clearly, we have $f(0) = -1$, $f(1) = 1$ and $f'(x) = kx^{k-1} + 1 > 0$. Thus, there is the unique ρ with $0 < \rho < 1$ such that $\rho^k + \rho - 1 = 0$. Here we assume that the root ρ can be found in a constant time for each k with $k > 1$. By using the Matlab software, we compute the ρ values for $k = 2, 3, \ldots, 10$, respectively. The detailed values are as follows:

$$\rho = \begin{cases} 0.618 & \text{when } k = 2, \\ 0.682 & \text{when } k = 3, \\ 0.725 & \text{when } k = 4, \\ 0.755 & \text{when } k = 5, \\ 0.778 & \text{when } k = 6, \\ \cdots & \\ 0.835 & \text{when } k = 10, \end{cases}$$

Set $\gamma = \frac{1}{\rho^k} = \frac{1}{1-\rho}$. Furthermore, we also have

$$\gamma = \begin{cases} 2.618 & \text{when } k = 2, \\ 3.145 & \text{when } k = 3, \\ 3.636 & \text{when } k = 4, \\ 4.082 & \text{when } k = 5, \\ 4.505 & \text{when } k = 6, \\ \cdots & \\ 6.098 & \text{when } k = 10, \end{cases}$$

Next, we show that the approximation ratio of algorithm A_3 for problem $1||\left(C_{\max}\right)^k + \sum_{J_j \in R} e_j$ is not greater than γ. Let σ be the schedule obtained from algorithm A_3 and let Z be the value of objective function of σ. Furthermore, we also assume that Z^* and Z_s^* are the optimal values of objective function for problems $1||\left(C_{\max}\right)^k + \sum_{J_j \in R} e_j$ and $1|\text{split}|\left(C_{\max}\right)^k + \sum_{J_j \in R} e_j$, respectively. We have the following theorem.

Theorem 5 $Z \leq \gamma Z_s^* \leq \gamma Z^*$.

Proof If no job is split in σ^*, then we output the schedule σ^*. Clearly, in this situation, σ^* is optimal for problem $1||\left(C_{\max}\right)^k + \sum_{J_j \in R} e_j$. Otherwise, we assume that J_{j^*} is the unique job which is split in σ^*. We distinguish two cases into our discussions.

If $y_{j^*} \geq \rho$, by algorithm A_3, then we have

$$\begin{aligned} Z &= \left(\sum_{j=1}^{j^*} p_j\right)^k + \sum_{j=j^*+1}^{n} e_j \\ &\leq \left(\frac{1}{y_{j^*}}\left(\sum_{j=1}^{j^*-1} p_j + y_{j^*} p_{j^*}\right)\right)^k + \left(1 - y_{j^*}\right) e_{j^*} + \sum_{j=j^*+1}^{n} e_j \\ &\leq \left(\frac{1}{\rho}\right)^k \left(\left(\sum_{j=1}^{j^*-1} p_j + y_{j^*} p_{j^*}\right)^k + \left(1 - y_{j^*}\right) e_{j^*} + \sum_{j=j^*+1}^{n} e_j\right) \\ &\leq \gamma Z_s^* \\ &\leq \gamma Z^*. \end{aligned}$$

If $y_{j^*} < \rho$, by algorithm A_3, then we have

$$\begin{aligned} Z &= \left(\sum_{j=1}^{j^*-1} p_j\right)^k + \sum_{j=j^*}^{n} e_j \\ &\leq \left(\sum_{j=1}^{j^*-1} p_j + y_{j^*} p_{j^*}\right)^k + \frac{1}{1-y_{j^*}}\left(\left(1 - y_{j^*}\right) e_{j^*} + \sum_{j=j^*+1}^{n} e_j\right) \\ &\leq \frac{1}{1-\rho}\left(\left(\sum_{j=1}^{j^*-1} p_j + y_{j^*} p_{j^*}\right)^k + \left(1 - y_{j^*}\right) e_{j^*} + \sum_{j=j^*+1}^{n} e_j\right) \\ &\leq \gamma Z_s^* \\ &\leq \gamma Z^*. \end{aligned}$$

This completes the proof of Theorem 5. □

Now, we continue to use the numerical example in Table 2 to illustrate the working of Algorithm A_3 for problem $1||\left(C_{\max}\right)^2 + \sum_{J_j \in R} e_j$. Note that, when $k = 2$, we have $\rho = \frac{\sqrt{5}-1}{2} \approx 0.618$ and $y_4 = \frac{1}{3} < \rho$. Thus, algorithm A_3 will accept the jobs J_1, J_2, J_3 and reject the jobs $J_4, \ldots, J_7$. That is, we have $Z = 8^2 + 340 = 404$. Note that, for problem $1|\text{split}|\left(C_{\max}\right)^k + \sum_{J_j \in R} e_j$, the optimal objective value is $Z_s^* = 400$. Thus, the objective value obtained by algorithm A_3 is very near the optimal value for $1|\text{split}|\left(C_{\max}\right)^k + \sum_{J_j \in R} e_j$. In fact, by the numerical example of algorithm DP1, algorithm A_3 outputs an optimal solution for problem $1||\left(C_{\max}\right)^2 + \sum_{J_j \in R} e_j$.

Finally, based on the dynamic programming algorithm and the rounding technique, we also obtain a fully polynomial-time approximation scheme (FPTAS) for problem $1||\left(C_{\max}\right)^k + \sum_{J_j \in R} e_j$. By Theorem 5, we have $Z^* \leq Z \leq \gamma Z^*$. For each job J_j with $e_j > Z$, we can conclude that J_j must be accepted in any optimal schedule. Otherwise, we have $Z^* \geq e_j > Z \geq Z^*$, a contradiction. If we modify the rejection cost of J_j such that $e_j = Z$, this does not change the optimal objective value. Thus, without loss of generality, we can assume that $e_j \leq Z$. The FPTAS for problem $1||\left(C_{\max}\right)^k + \sum_{J_j \in R} e_j$ is as follows.

Algorithm A_ϵ

Step 1. For any $\epsilon > 0$, set $M = \frac{\epsilon Z}{n\gamma}$. Given an instance I, we define a new instance I' by rounding the rejection cost of the jobs in I such that $e'_j = \lfloor \frac{e_j}{M} \rfloor M$, for $j = 1, \ldots, n$.

Step 2. Apply the dynamic programming algorithm DP1 to the instance I', we can obtain an optimal solution $\pi^*(I')$ for the instance I'.

Step 3. For each $j = 1, \ldots, n$, we replace the modified rejection cost e'_j by the original rejection cost e_j in $\pi^*(I')$. We can obtain a feasible solution π for the instance I.

Let Z_ϵ be the objective value of the schedule π obtained from A_ϵ. We have the following theorem.

Theorem 6 *$Z_\epsilon \leq (1+\epsilon)Z^*$ and the running time of A_ϵ is $O(\frac{n^3}{\epsilon})$.*

Proof Let $Z^*(I')$ be the optimal objective value of the schedule $\pi^*(I')$. From step 1 of A_ϵ, we have $e'_j \leq e_j < e'_j + M$. Thus, we have $Z^*(I') \leq Z^*$. Replace e'_j by e_j for each $j = 1, \ldots, n$, we have

$$Z_\epsilon \leq Z^*(I') + \sum_{j=1}^{n}(e_j - e'_j) \leq Z^* + nM \leq Z^* + \frac{\epsilon Z}{\gamma} \leq (1+\epsilon)Z^*.$$

Since $e_j \leq Z$ for $j = 1, \ldots, n$, we have $e'_j \leq Z$. Furthermore, we have $\sum_{j=1}^{n} e'_j \leq nZ$. Note that, since all e'_js are the multiple of M, all E values in DP1 are also the multiple of M. It follows that $kM = E \leq \sum_{j=1}^{n} e'_j \leq nZ$, i.e., $k \leq \frac{nZ}{M} = O\left(\frac{n^2}{\epsilon}\right)$. Thus, there are at most $O\left(\frac{nZ}{M}\right) = O\left(\frac{n^2}{\epsilon}\right)$ choices for all E values in DP1. We can conclude the running time of DP1 (also A_ϵ) is $O\left(\frac{n^3}{\epsilon}\right)$. This completes the proof of Theorem 6. □

5 Problem $Pm|\text{split}|\left(C_{\max}\right)^k + \sum_{J_j \in R} e_j$

In this problem, we assume that each job J_j can be split arbitrarily in $q > 1$ parts $J_j^1, J_j^2, \ldots, J_j^q$ such that $\sum_{i=1}^{q} p_j^i = p_j$, $\sum_{i=1}^{q} e_j^i = e_j$ and $e_j^i = \frac{e_j}{p_j} \cdot p_j^i$, where p_j^i and e_j^i are the corresponding processing time and rejection cost of the split part J_j^i, $i = 1, \ldots, q$. All split parts $J_j^1, J_j^2, \ldots, J_j^q$ of J_j can be viewed as q independent jobs so that they can be accepted or rejected independently. Furthermore, all accepted parts can be processed in parallel, i.e., some overlaps are allowed in the processing of these accepted parts.

Assume that π^* is an optimal schedule for problem $Pm|\text{split}|\left(C_{\max}\right)^k + \sum_{J_j \in R} e_j$. For each job J_j, we can assume that there is at most a split part which is processed on each machine M_i and there is also a split part which is rejected in π^*. Otherwise, if there are multiple split parts which are processed on machine M_i or rejected in π^*, then we can glue them together into a longer part and this does not increase the makespan and the total rejection cost. Thus, we can assume that $J_j^1, J_j^2, \ldots, J_j^m, J_j^{m+1}$ are all split parts of J_j in π^*, where J_j^i is the split part processed on machine M_i when $i = 1, \ldots, m$ and J_j^{m+1} is the split part which is rejected in π^*. Note that, if some part

J_j^i with $1 \le i \le m+1$ does not exist in π^*, then we can assume that J_j^i is a dummy part with $p_j^i = 0$ and $e_j^i = 0$. We have the following lemma.

Lemma 2 *For problem $Pm|split|\left(C_{\max}\right)^k + \sum_{J_j \in R} e_j$, there is an optimal schedule such that $p_j^1 = p_j^2 = \cdots = p_j^m$ holds for each job J_j, $j = 1, \ldots, n$.*

Proof We can glue all split parts $J_j^1, J_j^2, \ldots, J_j^m$ together into a longer part and split the longer part into m identical parts with a length of $\frac{\sum_{i=1}^m p_j^i}{m}$ which are processed in parallel. Clearly, the yielded makespan is exactly $\frac{\sum_{j=1}^n \sum_{i=1}^m p_j^i}{m}$, which is a lower bound on the optimal makespan. Thus, the obtained schedule is still optimal. Lemma 8 follows. □

By Lemma 8, the sub-schedules on the m machines are identical. That is, these m machines are equivalent to a single machine with a speed of m. Thus, problem $Pm|\text{split}|\left(C_{\max}\right)^k + \sum_{J_j \in R} e_j$ with the processing times $\{p_1, \ldots, p_n\}$ can be transformed into the corresponding problem $1|\text{split}|\left(C_{\max}\right)^k + \sum_{J_j \in R} e_j$ with the processing times $\left\{\frac{p_1}{m}, \ldots, \frac{p_n}{m}\right\}$. Consequently, by using algorithms A_1 and A_2, we can obtain an optimal schedule for problem $1|\text{split}|\left(C_{\max}\right)^k + \sum_{J_j \in R} e_j$ with $k < 1$ and problem $1|\text{split}|\left(C_{\max}\right)^k + \sum_{J_j \in R} e_j$ with $k > 1$, respectively. Thus, we have the following theorem.

Theorem 7 *Problem $Pm|split|\left(C_{\max}\right)^k + \sum_{J_j \in R} e_j$ can be solved in $O(n \log n)$ time.*

6 Problem $Pm||\left(C_{\max}\right)^k + \sum_{J_j \in R} e_j$

Problem $Pm||C_{\max}$ has been proved to be binary NP-hard when m is a fixed constant and strongly NP-hard when m is an arbitrary number. Note that problem $Pm||\left(C_{\max}\right)^k$ has the same computational complexity with problem $Pm||C_{\max}$. Thus, as an extension of problem $Pm||\left(C_{\max}\right)^k$, problem $Pm||\left(C_{\max}\right)^k + \sum_{J_j \in R} e_j$ is at least binary NP-hard when m is a fixed constant and strongly NP-hard when m is an arbitrary number.

For problem $Pm||C_{\max} + \sum_{J_j \in R} e_j$, Bartal et al. (2000) presented a dynamic programming algorithm. By a simple modification, we can obtain the dynamic programming algorithm for problem $Pm||\left(C_{\max}\right)^k + \sum_{J_j \in R} e_j$.

Let $f_j(t_1, \ldots, t_m)$ be the minimum value of the total rejection cost under the following constraints: (1) the jobs in consideration are $J_1, \ldots, J_j$; and (2) the maximum completion time on machine M_i is exactly t_i, $i = 1, \ldots, m$. We have the following dynamic programming algorithm for problem $Pm||\left(C_{\max}\right)^k + \sum_{J_j \in R} e_j$.

Dynamic programming algorithm DP2
The boundary conditions:

$$f_1(t_1,\ldots,t_m)=\begin{cases} e_1, & \text{if } t_1=\cdots=t_m=0;\\ 0, & \text{if } t_i=p_1 \text{ for some } i \text{ and } t_j=0 \text{ for each } j\neq i;\\ +\infty, & \text{otherwise.}\end{cases}$$

The recursive function:

$$f_j(t_1,\ldots,t_m)=\min\{f_{j-1}(t_1,\ldots,t_m) +e_j,\ \min_{1\le i\le m}\{f_{j-1}(t_1,\ldots,t_{i-1},t_i-p_j,t_{i+1},\ldots,t_m)\}\}.$$

The optimal value is given by

$$\min\left\{f_n(t_1,\ldots,t_m)+\max\left(t_1^k,\ldots,t_m^k\right):0\le t_1,\ldots,t_m\le\sum_{j=1}^{n}p_j\right\}.$$

Theorem 8 *Algorithm DP2 solves problem* $Pm||\left(C_{\max}\right)^k+\sum_{J_j\in R}e_j$ *in* $O\left(n(\sum_{j=1}^{n}p_j)^m\right)$. *time.*

Note that, when m is a fixed constant, algorithm DP2 is pseudo-polynomial-time algorithm for problem $Pm||\left(C_{\max}\right)^k+\sum_{J_j\in R}e_j$. Thus, problem $Pm||\left(C_{\max}\right)^k+\sum_{J_j\in R}e_j$ is exactly binary NP-hard when m is a fixed constant.

7 Conclusions and future research

In this paper we study several scheduling problems with rejection on $m\ge 1$ identical machines. The objective is to minimize the sum of the k-th power of the makespan of accepted jobs and the total rejection cost of rejected jobs. We also introduce the conception of "job splitting" in our problems. First, we consider the single machine scheduling problem, i.e., $m=1$. When job splitting is allowed, we propose an $O(n\log n)$-time optimal algorithm. When job splitting is not allowed, we show that this problem is polynomially solvable when $k\in(0,1)$ and it becomes binary NP-hard when $k>1$. Furthermore, for the NP-hard problem, we propose a pseudo-polynomial dynamic programming algorithm and a fully polynomial-time approximation scheme (FPTAS). Finally, we also extend our problems and some results to $m\ge 2$ identical parallel machines.

For future research, it is challenging to develop some approximation algorithms for the parallel machine scheduling problems. For parallel machine scheduling problem $P||C_{\max}+\sum_{J_j\in R}e_j$, Bartal et al. (2000) provided an FPTAS when m is fixed and a PTAS when m is arbitrary. Thus, it is interesting to borrow their idea to obtain an

FPTAS or a PTAS for our problem $P||\left(C_{\max}\right)^k + \sum_{J_j \in R} e_j$. In addition, it is also interesting to consider the on-line scheduling problems even when $m = 1$.

Funding This work was supported by the National Natural Science Foundation of China under Grant Numbers 12271491, 11971443 and 11901168.

Data Availability Not applicable.

Declarations

Conflict of interest The authors report no conflict of interest.

References

Atsmony M, Mosheiov G (2023) Scheduling to maximize the weighted number of on-time jobs on parallel machines with bounded job-rejection. J Sched 26:193–207

Bartal Y, Leonard S, Spaccamela AM, Sgall J, Stougie L (2000) Multi-processor Scheduling with Scheduling problems with rejection. SIAM J Discret Math 13:64–78

Chen RX, Li SS (2020) Minimizing maximum delivery completion time for order scheduling with rejection. J Comb Optim 40:1–21

Garey MR, Johnson DS (1979) Computers and intractablity: a guide to the theory of NP-completeness. Freeman, San Francisco, CA

Hermelin D, Pinedo M, Shabtay D, Talmon N (2019) On the parameterized tractability of a single machine scheduling with rejection. Eur J Oper Res 273:67–73

Hoogeveen H, Skutella M, Woeginger GJ (2003) Preemptive scheduling with rejection. Math Program 94:361–374

Ishii H, Nishida T (1986) Two machine open shop scheduling problem with controllable machine speeds. J Oper Res Soc Jpn 29:123–131

Ishii H, Mashuda T, Nishida T (1987) Two machine mixed shop scheduling problem with controllable machine speeds. Discret Appl Math 17:29–38

Korte B, Vygen J (2018) Combinatorial optimization: theory and algorithms, 6th edn. Springer, Berlin

Koulamas C, Kyparisis GJ (2023) Two-stage no-wait proportionate flow shop scheduling with minimal service time variation and optional job rejection. Eur J Oper Res 305:608–616

Liu PH, Lu XW (2020) New approximation algorithms for machine scheduling with rejection on single and parallel machine. J Comb Optim 40:929–952

Liu ZX (2020) Scheduling with partial rejection. Oper Res Lett 48:524–529

Ma R, Guo SN (2021) Applying "Peeling Onion" approach for competitive analysis in online scheduling with rejection. Eur J Oper Res 290:57–67

Mor B, Mosheiov G, Shapira D (2020) Flowshop scheduling with learning effect and job rejection. J Sched 23:631–641

Mor B, Shabtay D (2022) Single-machine scheduling with total late work and job rejection. Comput Ind Eng 169:108168

Nowicki E, Zdrzalka S (1990) A survey of results for sequencing problems with controllable processing times. Discret Appl Math 26:271–287

Oron D (2021) Two-agent scheduling problems under rejection budget constraints. Omega 102:102313

Shabtay D, Gaspar N, Kaspi M (2013) A survey on offline scheduling with rejection. J Sched 16:3–28

Shabtay D, Steiner G (2007) A survey of scheduling with controllable processing times. Discret Appl Math 155:1643–1666

Shakhlevich NV, Strusevich VA (2006) Single machine scheduling with controllable release and processing times. Discret Appl Math 154:2178–2199

Van Wassenhove LN, Baker KR (1982) A bicriterion approach to time/cost trade-offs in sequencing. Eur J Oper Res 11:48–54

Xing WX, Zhang JW (2000) Parallel machine scheduling with splitting jobs. Discret Appl Math 103:259–269

Zhang LQ, Lu LF, Yuan JJ (2009) Single machine scheduling with release dates and rejection. Eur J Oper Res 198:975–978
Zhang YZ (2020) A survey on job scheduling with rejection (in Chinese). OR Trans 24(2):111–130